Emergency Response Decision-Support System

Manasseh Ophalfens (s0231648) · Artificial Intelligence, UAntwerpen · 12/11/2026

Repository: https://github.com/Manaci4466/Emergency-Response-System-Assignment

Approach & implementation

The system turns noisy road reports and historical traffic observations into expected travel times, finds routes with A*, and assigns responders with a constraint solver each time new incidents become known. `run.py` writes `plan.json` and `statistics.txt` and then checks the plan with an independent validator. Roads are **directed**: A→B and B→A never share reports or observations.

Bayesian road model (`bayes.py`, Notes 6). Each road has its own Bayesian network Weather → Status → {Citizen, Police, Sensor}, solved by enumeration: $P(S \mid w, r) \propto P(S|w) \cdot \Pi\, P(r_i|S)$. The reports may be multiplied because Status is their only parent (conditionally independent given S); P(w) is the same for every status and cancels in the normalisation. For roads without reports the weather is summed out, giving the prior Open 0.685 / Congested 0.18 / Blocked 0.135. Example: small N1→N2 (Rain; Blocked, Blocked, Congested) gets P(Blocked) = 0.527 ≥ τ, so it is avoided, while N2→N1 (all Congested) stays usable.

HMM traffic model (`hmm.py`, Notes 8). Hidden state = true traffic level, evidence = observed level. The forward algorithm alternates time elapse and observation and normalises every step, which prevents numerical underflow. We filter instead of running Viterbi because planning needs the distribution of the *current* level, not the most likely history. The traffic at t = 0 is the filtered belief plus one prediction step, converted into an expected multiplier (Low 1.0, Medium 1.25, High 1.5; roads without observations ×1.0).

Cost model and A* (`cost_model.py`, `search.py`, Notes 1). Expected time = distance/speed × status multiplier $(P(O)\cdot 1 + P(C)\cdot 1.5)/(P(O)+P(C))$ × traffic multiplier; a road is unusable if P(Blocked) ≥ τ = 0.5. A* (graph search, closed set) uses h = straight-line distance / fastest road speed: admissible because every road is at least as long as the straight line and every multiplier is ≥ 1, and consistent by the triangle inequality, so the closed set never discards a better path. The program checks the first condition on each scenario and otherwise uses h = 0 (uniform-cost search).

CSP allocation (`allocation.py`, Notes 2). Each round (the incidents with the same timestamp) is a CSP: variables = incidents, domains = capable responders that can reach them. A busy responder is queued and starts its next job from its last incident once that job ends. Deadlines are soft (late help beats no help), so forward checking is adapted: an incident with no on-time responder left does not fail the branch, it adds its priority weight (Critical 8, High 4, Medium 2, Low 1) to a lower bound. Branch and bound minimises first the priority-weighted lateness and then the total response time. MRV picks the incident with the fewest on-time responders. Because the assignment order also fixes the job order of a shared responder, MRV here changes the *answer*, not only the speed: without it, R10 in medium handles I2 before the Critical I3, which then arrives late. The bound is valid only because A* returns true shortest paths: admissible h → optimal paths → valid bound → correct branch and bound. Rounds are solved in time order with only the incidents known at that moment (no clairvoyance) and committed immediately.

LLM collaboration

I used **Claude chat** (claude.ai) for concept explanations, the design and a build guide, and as a **reviewer** after every phase: it re-ran the tests, checked numbers by

hand and mutation-tested the code. **Claude Code** (Claude Opus 5.5, in the terminal next to PyCharm) implemented each phase with its tests; the reviewer's findings became my next prompt (Appendix A: Prompt 1 and key follow-ups; full log in `PROMPTS.md`; "P6" = Prompt 6). Every numerical value an LLM produced was verified with a test or a hand calculation before I relied on it.

Interaction	What I did and why
Phase 1: CPT-orientation test on a diagonal entry and a symmetric table (P6)	**Modified**: a test that reads the same either way round can never fail; replaced by an asymmetric entry (0.06 vs 0.08).
Phase 3: HMM tests (P9)	**Modified**: my mutation test showed that applying a transition before observation 1 passed all tests (the HMM forgets its start); added a one-observation test. The LLM's eigenvalue claim (0.7) was wrong (0.742).
Phase 4: "A* = UCS" as the only optimality check (P10)	**Modified**: both share one function, so a shared bug passes; added differential tests against an independent Floyd–Warshall.
Phase 5: my reviewer's predicted test numbers (P12)	**Verified in both directions**: Claude Code checked them against the cost model first and found the reviewer's greedy arrival (147.4 s) wrong (89.6 s); the reviewer's suggested assertion tested the wrong thing and was replaced.
Phase 4 commit	**LLM process error**: it committed while tests were failing (`pytest | tail` hides pytest's exit code; a stale `.pyc` from its own mutation check caused the failures), then explained it and changed its process.

Testing & results

`python -m pytest` runs **110 tests** (plus an opt-in, slow mutation sweep), using **hand-calculated values** (N1→N2 posterior 0.014 / 0.459 / 0.527; the classic umbrella HMM exactly with fractions, 2/5 and 15/128; the allocation of small's first round), **differential testing** (A*, UCS and an independent Floyd–Warshall agree on every node pair; the CSP matches exhaustive brute force on the first round of every scenario) and **property tests** (consistent heuristic, no clairvoyance). The **validator** reads only `plan.json` and the raw files and never imports the planner; all 13 injected plan faults are caught with exactly the expected error. A **mutation sweep** applies 35 typical bugs from every module to a copy of the project: **35 of 35 are caught** (its first run exposed an untested τ boundary). Testing works in layers: the validator shares the cost model, so an error in the model itself (e.g. Congested ×1.0 instead of ×1.5) still produces "valid" plans; only the hand-calculated unit tests catch that. In the results below, deadlines met = on-time arrivals / *all* incidents (unhandled = missed), the average response time (arrival − timestamp) is over handled incidents, and weighted missed = Σ priority weights of late or unhandled incidents. All values are **expected** seconds under the model, not observed ground truth.

Scenario	Policy	Handled	Responders used	Avg response (s)	Deadlines met	Weighted missed	Critical on time
1 (small)	CSP / greedy	7/7	4 / 4	112.8 / 112.2	28.6% / 28.6%	**24** / 28	**1/2** / 0/2
2 (medium)	CSP / greedy	12/12	8 / 8	34.6 / 37.2	91.7% / 91.7%	**4** / 8	**6/6** / 5/6
3 (large)	CSP / greedy	20/20	8 / 8	1319.2 / 1319.2	65.0% / 65.0%	36 / 36	5/8 / 5/8

The required deadline percentage weights all incidents equally, which hides that the CSP keeps a Critical incident on time where greedy does not. In small, several deadlines are missed by only 1–5 s, so they depend on the model's assumptions.

Experiment (`python -m src.experiments`; CSP, deadlines met unless stated)	small	medium	large
τ = 0.1 (closes every unreported road)	14.3%, 6 unhandled	33.3%, 8 unhandled	20.0%, 16 unhandled
τ = 0.6 (opens the roads at P(Blocked) 0.527)	57.1%	91.7%	65.0%
Unreported roads assumed open	85.7%	100.0%	75.0%
Unobserved traffic at the stationary multiplier	28.6% (all observed)	33.3%	45.0%
Monte Carlo, τ 0.5: P(≥1 planned route blocked)	0.68	0.64	0.97
Monte Carlo, τ 0.5 / 0.6: sampled mean deadlines met	36.2% / 35.1%	78.2% / 78.2%	43.1% / 42.1%

The results depend more on the modelling assumptions than on the algorithm: medium falls from 91.7% to 33.3% when its unobserved roads get the stationary traffic multiplier, because some of its deadlines are only 20 s after the incident appears. **Monte Carlo** sampling (prior sampling, Notes 6) draws 5000 possible worlds, sampling each road's true status and traffic level from our beliefs, and replays the fixed plan in each. Raising τ to 0.6 doubles small's *planned* deadlines met, but the sampled value drops slightly (36.2% → 35.1%), because the extra roads are blocked in 52.7% of the worlds; this supports τ = 0.5. R1's route to the Critical I2 crosses two unreported roads and is blocked in 24.9% of the worlds (analytically $1 - 0.865^2 = 25.2\%$).

Critical reflection

Limitations. These limitations became clear during different phases of the project. Every number in this report is an *expected* value under my own model. A "valid" plan only respects the hard constraints under my beliefs: the Monte Carlo replay shows that at least one planned route is blocked in 64–97% of the sampled worlds, and that the route to a Critical incident in small is blocked in one world out of four. The beliefs are computed once at t = 0 and used for the whole simulation, assignments are never re-planned when a more urgent incident arrives, reports are assumed independent given the road status (false if citizens copy the police), and the multipliers are chosen by me, not learned from data. The experiments show that these assumptions move the results more than the choice of algorithm.

The LLMs. They made me much faster and explained the theory well, and the reviewer–implementer setup found edge cases I would have missed. They also misled me in convincing ways: a test that could never fail, a wrong eigenvalue, a commit made while tests were failing, one scenario's stationary multiplier applied to all three scenarios, and two wrong predictions from the reviewing LLM. I caught each one only because I treated every claim as something to test: hand calculations, independent scripts and mutation testing. The main lesson is that a passing test is only evidence if it can fail.

Next steps (suggested by Claude Code; I adopted them after checking their reasoning). (1) Rolling-horizon re-planning: re-solve when a new incident arrives and compare the weighted missed and the Monte Carlo deadlines met on all three scenarios. (2) Risk-aware routing that penalises P(Blocked) instead of a hard τ, measured by the Monte Carlo P(route blocked) and deadlines met. (3) Time-varying traffic, which needs a time unit for the observations, measured the same way.

Appendix A: LLM prompts

Full log with responses and what I did: `PROMPTS.md` (numbers below match it).

Prompt 1 (first substantive prompt, Claude chat, verbatim):

> I need to complete this assignment but I do not know exactly how. For this prompt I am not going to explain everything thorougly, but I just want you to work with me and give me a guide that maximalizes both the project generated as well as my understanding of it. I still need to add all the scenarios so wait a bit before generating or doing anything. When I send a prompt with only 'ready' in it. You may begin!

Prompt 6 (phase 1 review, Claude Code, paraphrased):

> Fixes before phase 2: The CPT orientation test cannot fail: Police[Blocked][Blocked] is diagonal and the citizen table is symmetric. Test an asymmetric entry instead (Police["Open"]["Congested"] == 0.06. flipped it would be 0.08) and explain in a comment why the old check proved nothing. […]

Prompt 9 (phase 3 review, Claude Code, paraphrased):

> Add a test pinning the convention initial = P(X1): small model, ONE observation "Low" must give B ∝ emission·initial = (0.941, 0.0554, 0.0037). Explain in a comment why the existing tests cannot catch a transition applied before observation 1 […]. Prove it: temporarily apply the transition before observation 1, show only the new test fails, then revert. 2. Fix the comment: the 2nd eigenvalue of T is 0.742 […]

Prompt 10 (phase 4 review, Claude Code, paraphrased):

> A* and UCS share _best_first, so a bug inside it makes both wrong the same way and "A* == UCS" still passes […]. In the every-pair test: (a) recompute each returned path's cost by summing costs[(a,b)].cost over its roads […]; (b) compare with an independent all-pairs reference written in the test itself (Floyd–Warshall).

Prompt 12 (phase 5 review, Claude Code, paraphrased):

> assert in the brute-force test that the CSP explores fewer nodes than brute force evaluates plans (removing pruning currently goes unnoticed)

Claude Code showed that this assertion holds even without pruning and replaced it with a pruning-on vs pruning-off test.

Prompt 17 (phase 8 review, Claude Code):

> Experiment 3 bug: the stationary multiplier is computed from small's traffic model and applied to all scenarios. Medium has a different traffic_probabilities.json (its stationary multiplier is 1.25). Compute it per scenario and show it per row. […]